arXiv:2408.15518v2 [cs.CL] 3 Sep 2024

Squid: Long Context as a New Modality for Energy-Efficient On-Device Language Models

Wei Chen
Nexa AI
Sunnyvale, CA 94086
alexchen@nexa4ai.com

Zhiyuan Li
Nexa AI
Sunnyvale, CA 94086
zack@nexa4ai.com

Shuo Xin
Nexa AI
Sunnyvale, CA 94086
shuo@nexa4ai.com

Yihao Wang
Nexa AI
Sunnyvale, CA 94086
ethan@nexa4ai.com

Abstract

This paper presents Squid, a novel decoder-decoder architecture for energy-efficient processing of long contexts in language models. Our approach addresses the significant energy consumption and latency challenges inherent in on-device models. Squid employs a compact 0.5B parameter decoder to distill extensive contextual information into a memory embedding, substantially reducing the input length for the primary 7B parameter decoder model. Inspired by vision-language models, we repurpose the image embedding projector to encode long textual contexts, effectively treating extended context as a distinct modality. This innovative method enables processing of substantially longer contexts without the typical computational overhead associated with extended input sequences. Empirical evaluations demonstrate a 10-fold improvement in energy efficiency and a 5-fold reduction in latency compared to conventional full-length context processing methods without losing quality of the response. Our work contributes to the development of more sustainable and scalable language models for on-device applications, addressing the critical need for energy-efficient and responsive AI technologies in resource-constrained environments while maintaining the accuracy to understand long contexts. This research has implications for the broader field of natural language processing, particularly in the domain of efficient model design for resource-limited settings. By enabling more sophisticated AI capabilities on edge devices, Squid paves the way for advanced language processing in a wide range of applications where computational resources are at a premium. The Squid model is publicly available at https://huggingface.co/NexaAIDev/Squid.

1 Introduction

The rise of on-device language models has become increasingly crucial in our interconnected world, offering enhanced privacy, reduced latency, and offline functionality[1, 2, 3, 4, 5, 6, 7]. However, these models face significant challenges, particularly in energy consumption and processing speed when handling long contexts. Battery life on mobile devices is a critical concern, as complex language processing tasks can rapidly deplete power resources, limiting the practical utility of on-device AI applications. This energy constraint is further exacerbated when processing long contexts, which require more computational resources and memory usage. Moreover, the latency introduced by processing extensive input sequences can severely impact user experience, especially in real-time applications such as voice assistants or interactive chatbots. Consequently, there is an urgent need

for innovative approaches that maintain the accuracy and capability of language models while significantly reducing their energy footprint and improving response times.

To address these challenges, various approaches have been developed to mitigate the context length problem in large language models (LLMs). Retrieval-Augmented Generation (RAG)[8] has emerged as a prominent solution, incorporating an external retrieval component to search for relevant information, thereby allowing the model to handle extensive knowledge without storing all information in its parameters. Recent advancements, such as the LongRAG[9] framework, have further improved this approach by balancing the workload between retriever and reader, enabling the processing of much larger token inputs. Another effective strategy focuses on optimizing the key-value (KV) cache. Techniques like chunk-wise KV cache compression and swapping have been implemented to minimize context-switching overhead and enable efficient state maintenance across multiple invocations. The LLMaaS (Language Models as a Service)[10] paradigm exemplifies this approach by integrating LLMs as system services on mobile devices, employing stateful execution to maintain persistent states and reduce memory usage. While these methods have shown promise, they often involve trade-offs between context length, model performance, and computational efficiency, highlighting the need for more holistic solutions.

Some other works has made efforts on directly reduce the length of the context to lower computational costs[12, 13, 14]. Although these approaches aim to reduce computational costs via context compression, this step itself can still introduce overhead, and they do not address the alignment issue between the compressed context and the original text.

In response to these challenges, we introduce Squid, a novel decoder-decoder architecture designed specifically for energy-efficient processing of long contexts in language models. We were inspired by recent works on Vision-Language Models (VLMs) [11, 16, 17, 18, 19], which demonstrate that model performance can benefit from specially designed multi-stage training procedures. Our approach utilizes a small 0.5B decoder to distill extensive contextual information into several memory tokens, significantly reducing the input length for the primary 7B decoder model. Drawing inspiration from vision-language models, we repurpose the image embedding projector to encode long textual contexts, effectively treating extended context as a distinct modality. This innovative method enables the processing of substantially longer contexts without incurring the typical computational overhead associated with extended input sequences. By doing so, Squid achieves an impressive 10-fold improvement in energy efficiency and a 5-fold improvement in latency compared to conventional full-length context processing methods. Our work contributes to the development of more sustainable and scalable language models for on-device applications, addressing the critical need for energy-efficient and fast AI technologies in resource-constrained environments while maintaining the accuracy to understand long contexts. This breakthrough has far-reaching implications for the deployment of sophisticated AI capabilities on edge devices, potentially revolutionizing fields such as mobile computing, IoT, and wearable technology.

2 Related Works

Prompt compression Prompt compression has emerged as a critical area of research to address challenges associated with long context inference in LLMs. Existing methods can be divided into three main categories: token pruning, abstractive compression, and extractive compression. Token pruning techniques, such as LongLLMLingua [35] and Selective-Context [34], aim to reduce prompt length by removing tokens of less importance. Abstractive compression methods, including RECOMP [33] and Prompt-SAW [32], utilize summarization techniques to condense the original context. Extractive compression methods, like RECOMP's extractive component and document rerankers [31, 30, 29]), select relevant documents, sentences, or phrases from the original context. These approaches can be further classified as query-aware or query-agnostic, depending on whether they tailor compression based on the specific question or task. Some other works directly compress or distill the context to lower computational costs. AutoCompressor[12] recursively compresses long contexts into compact summary vectors. CEPE [41] introduced parallel encoding to extend context window. Tan et al. address long context by offline learning in LLoCO [42]. StreamingLLM[44] and unlimiformer [43] attempt to change the attention mechanism to achieve a longer context window. REPLUG [45] uses a retrieval model to separately process contexts. Mu et al. proposed Gist tokens[13], which compress context length through context distillation and modifications to the attention mask. ICAE[14] uses an encoder, fine-tuned from an LLM via LoRA[15], to compress the context and employs multi-stage

training to enhance model performance. Early works also attempts to revise the LLM structure for longer context [46, 47, 48, 49, 50, 51, 52]. Despite growing interest in prompt compression, there has been a lack of standardized analysis comparing different methods across various tasks and compression ratios. This has led to conflicting results and makes it challenging for practitioners to choose the appropriate method for their specific applications. Our work aims to bridge this gap by providing a more comprehensive characterization and evaluation of different prompt compression methods across a range of tasks and compression rates.

Multimodal model Multimodal large language models (MLLMs)[11, 16, 17, 18, 19] represent a significant leap in AI technology, enhancing the abilities of conventional LLMs by enabling the simultaneous processing and analysis of multiple modalities. MLLMs generally comprise three key elements: a modality encoder, a projector, and an LLM. The modality encoder is responsible for processing data from different modalities such as text, images, video, and audio. Encoders such as ViT [20] or CLIP-ViT [27] are commonly used for visual data, while ConFormer [26] or HuBERT[25] may be utilized for audio data. For 3D point cloud data, encoders like ULIP-2 [24] have been developed. The projector plays a critical role in aligning data from different modalities with the LLM. This can be achieved through various mechanisms such as linear projectors, cross-attention mechanisms, Q-Former[21], or P-Former. The projector's role is to map the features extracted by the modality encoders into the LLM's embedding space, facilitating a unified representation across different modalities. The LLM, backbone of the MLLM, provides the core language comprehension and generation capabilities. Popular LLM architectures like Vicuna[36] or Llama 2[37] are often utilized for this purpose. The development of MLLMs is typically a two-phase process: initial pre-training of the individual components (LLM and modality encoders) separately, followed by integration through further training using mixed multimodal data. This approach allows MLLMs to leverage the strengths of each modality while maintaining a cohesive understanding across different types of input.

On-device model deployment Model deployment frameworks for on-device LLMs are critical for ensuring efficient execution across different hardware platforms. Dedicated frameworks like Llama.cpp [1], MNN [23], PowerInfer [6], ExecuTorch [4], and MediaPipe [3] focus on optimizing inference on local devices, supporting various hardware architectures and quantization techniques for efficiency. These frameworks leverage the capabilities of CPUs, GPUs, and other specialized hardware like neural processing units (NPUs) and digital signal processors (DSPs) to ensure optimal performance and resource utilization. Edge-cloud frameworks like MLC-LLM [2], VLLM [22], and OpenLLM [5] by BentoML enable flexible deployment options across both local devices and cloud environments, integrating advanced quantization and memory management techniques to balance computation load and maintain high throughput and efficiency. These strategies collectively enhance the feasibility and performance of on-device LLM deployments,catering to a wide range of applications and hardware limitations.

3 Methodology

This section outlines our approach to developing the Squid model for efficient long-context processing. We describe the novel decoder-decoder architecture, the implementation of memory tokens, our multi-stage training process, and the dataset used for training and evaluation. These components collectively address the challenges of energy efficiency and latency in on-device language models while maintaining long-context understanding capabilities.

3.1 Long Context as a Novel Modality

In our model architecture design, we introduce an innovative decoder-decoder framework for the Squid model, conceptualizing long context as a novel modality. This architecture comprises two decoders of disparate sizes: a smaller decoder π_s with 0.5B parameters, and a larger decoder π_l with 7B parameters. The smaller decoder π_s serves to transform information from the extensive context, while the larger decoder π_l primarily focuses on comprehending and generating responses to the current query. Figure 1 illustrates this architecture.

It is important to note that the text encoder depicted in Figure 1 is, in fact, a model based on transformer decoder architectures, specifically derived from Qwen2 0.5B[38]. The main decoder is based on Qwen2 7B. During the inference stage, we process the user's query $\mathcal{Q}$ and the context

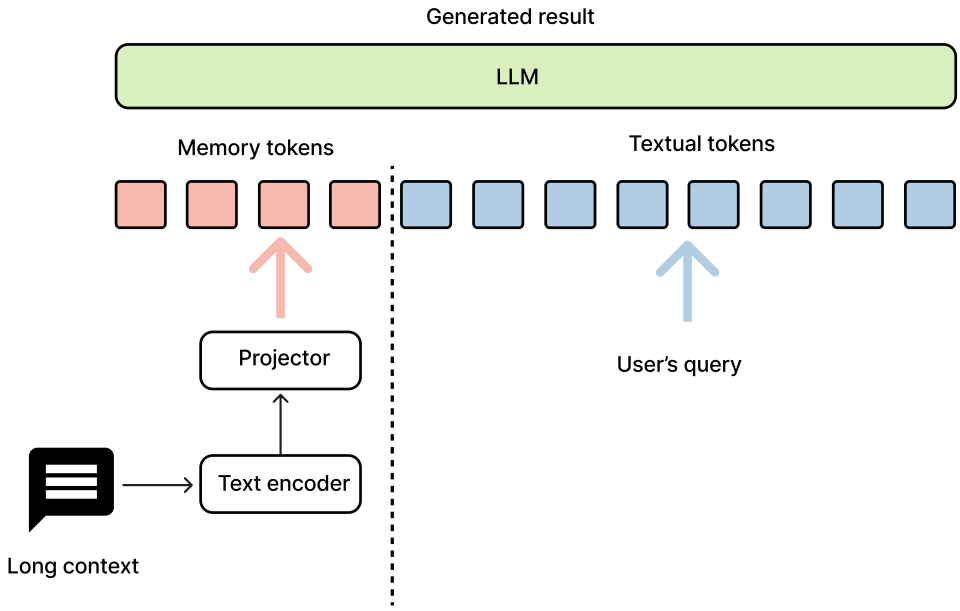

Figure 1: The model architecture of the Squid model, with three different components, the text encoder which is a model with transformer decoder architecture. The projector is to convert the embedding information after text encoder into the embedding that can be understood by the main LLM, which is another transformers decoder model.

$\mathcal{C}$, where typically $|\mathcal{Q}| \ll |\mathcal{C}|$, as is common in multi-round conversations or retrieval-augmented generation (RAG) scenarios.

Analogous to vision-language models, we incorporate a projector Φ to transform the embedding information post-text encoding into context token embeddings suitable for input into the main decoder. The projector Φ is implemented as a multi-layer perceptron (MLP), bridging the different embedding dimensions of the text encoder (896 for Qwen2 0.5B) and the main decoder (3584 for Qwen2 7B).

For the text encoder component, we utilize Qwen2 0.5B, denoted as π_s, which employs a transformer decoder architecture. The primary function of π_s is to convert the context $\mathcal{C}$ into an embedding representation that can be further processed by the main decoder. This process can be formally expressed as:

$$\mathcal{M} = \pi_s(\mathcal{C}) \tag{1}$$

Let L denote the context length. If we send the context directly into main decoder π_l, the size of the embedding to be processed would be $L \times 3584$. By first passing the context $\mathcal{C}$ through π_s, we aim to reduce the embedding size to N, with a compression rate $\rho = L/N$. Our experiments demonstrate that ρ can reach up to 8 without compromising the quality of the final response, compared to directly inputting the entire context and query into the main decoder model.

The complete process can be described by the following equations:

$$\mathcal{M} = \pi_s(\mathcal{C}) \tag{2}$$
$$\mathcal{E} = \Phi(\mathcal{M}) \tag{3}$$
$$\mathcal{R} = \pi_l(\mathcal{Q}, \mathcal{E}) \tag{4}$$

where $\mathcal{R}$ represents the generated response.

This decoder-decoder architecture offers several advantages. Firstly, it enables efficient processing of long contexts by using π_s to compress the context information, significantly reducing the computational burden on π_l. Secondly, the separate processing of context allows the model to treat

it as a distinct modality, similar to how vision-language models handle image inputs. Lastly, this architecture provides flexibility, allowing for easy adaptation to various tasks involving long contexts, such as multi-turn dialogues or document-based question answering.

3.2 Memory Tokens

To facilitate the extraction of information from long contexts using the text encoder model π_s, we introduce the concept of memory tokens. This approach involves augmenting the tokenizer with a set of special tokens, denoted as $[\text{memory_i}]_{i=0}^{N}$, and expanding the embedding space of π_s accordingly. These additional tokens serve to capture a latent representation of the long context $\mathcal{C}$. The procedure can be formalized as follows: Let $(c_1, c_2, ..., c_L)$ be the original context of length L. We append N memory tokens, resulting in an augmented context $\mathcal{C}$:

$$\mathcal{C}' = (c_1, c_2, ..., c_L, [\text{memory_0}], [\text{memory_1}], ..., [\text{memory_N-1}]) \tag{5}$$

The augmented context $\mathcal{C}$ has a total length of $L + N$. We then process $\mathcal{C}$ through the text encoder model π_s:

$$\mathcal{Z} = \pi_s(\mathcal{C}') \in \mathbb{R}^{(L+N)\times d_s} \tag{6}$$

where $\mathcal{Z}$ is the resulting embedding matrix and d_s is the embedding dimension of π_s. The latent representation $\mathcal{M}$ of the context is obtained by extracting the embeddings corresponding to the memory tokens:

$$\mathcal{M} = \mathcal{Z}_{L+1:L+N} \in \mathbb{R}^{N\times d_s} \tag{7}$$

This matrix $\mathcal{M}$ encapsulates the condensed information from the long context, which can be efficiently processed by subsequent components of our framework. The use of memory tokens allows for a flexible and compact representation of extensive contextual information, potentially improving the model's ability to handle long-range dependencies and reducing computational overhead in downstream tasks.

3.3 Multi-stage Training

Our training process for the Squid model comprises three distinct stages: restoration training, continual training, and instruction fine-tuning. This multi-stage approach is designed to progressively enhance the model's ability to handle long contexts and generate appropriate responses.

3.3.1 Restoration Training

In the initial stage, we focus on the model's ability to reconstruct information from compressed embeddings. Given a context $\mathcal{C}$, we first compress it using the text encoder π_s and projector Φ:

$$\mathcal{E} = \Phi(\pi_s(\mathcal{C})) \tag{8}$$

The main decoder π_l is then trained to restore the original context from this compressed representation:

$$\hat{\mathcal{C}} = \pi_l(\mathcal{E}) \tag{9}$$

The objective is to minimize the difference between $\hat{\mathcal{C}}$ and $\mathcal{C}$, ensuring that π_l can effectively reconstruct the original information from the compressed embedding. We can incorporate special token or prompts to drive the restoration.

3.3.2 Continual Training

The second stage focuses on enhancing the model's capability to generate coherent continuations of partial contexts. We partition the context $\mathcal{C}$ into two segments, $\mathcal{C}_1$ and $\mathcal{C}_2$. The model is trained to generate $\mathcal{C}_2$ given the compressed representation of $\mathcal{C}_1$:

$$\mathcal{E}_1 = \Phi(\pi_s(\mathcal{C}_1)) \tag{10}$$

$$\hat{\mathcal{C}}_2 = \pi_l(\mathcal{E}_1) \tag{11}$$

The training objective is to minimize the discrepancy between $\hat{\mathcal{C}}_2$ and $\mathcal{C}_2$, thereby improving the model's ability to generate contextually appropriate continuations.

3.3.3 Instruction Fine-tuning

In the final stage, we fine-tune the model on instruction-following tasks. Given a context $\mathcal{C}$ and a query $\mathcal{Q}$, the model is trained to generate an appropriate response $\mathcal{R}$:

$$\mathcal{E} = \Phi(\pi_s(\mathcal{C})) \tag{12}$$

$$\hat{\mathcal{R}} = \pi_l(\mathcal{Q}, \mathcal{E}) \tag{13}$$

The objective is to minimize the difference between $\hat{\mathcal{R}}$ and the ground truth response $\mathcal{R}$, enhancing the model's ability to generate relevant and accurate responses to queries within the given context.

3.3.4 Comparison with Vision-Language Model Training

To elucidate the similarities and differences between our approach and vision-language model training processes, we provide a comparison with LLaVA (Large Language and Vision Assistant) in Table 1.

Training Stage	Squid (Our Model)	LLaVA[11]
Initial Training	Restoration Training: Reconstruct original context from compressed embeddings	Feature Alignment: Align image embeddings with text embeddings
Intermediate Training	Continual Training: Generate context continuations from partial compressed contexts	Visual Instruction Tuning: Fine-tune on image-text pair datasets
Final Training	Instruction Fine-tuning: Generate responses to queries given compressed contexts	Conversation Fine-tuning: Train on multi-turn conversations involving images

Table 1: Comparison of training stages between Squid and LLaVA

While both approaches employ multi-stage training, they differ in their specific objectives. Our model focuses on handling long textual contexts, whereas LLaVA emphasizes the integration of visual and textual information. Nevertheless, both methodologies aim to enhance the model's ability to process multimodal inputs and generate contextually appropriate responses.

3.4 Dataset

For the training and evaluation of our Squid model, we curated a diverse and comprehensive dataset tailored to each stage of our multi-stage training process. The dataset was designed to enhance the model's capacity to handle long contexts, generate coherent continuations, and respond accurately to user queries across various domains. For the restoration training stage, we compiled a dataset of 100K context samples sourced from diverse domains to ensure broad coverage and generalizability. The continual training stage utilized an additional 100K context samples, distinct from those used in the restoration training, specifically curated to facilitate the model's learning of coherent context continuation.

The instruction fine-tuning stage employed a comprehensive dataset of 1M question-answer pairs coupled with relevant contexts. This dataset encompassed a wide array of domains, including general knowledge, specific subject areas, and real-world scenarios. The contexts varied in length and complexity, to challenge the model's ability to extract and utilize relevant information effectively across 20 different domains. To ensure the quality and diversity of our datasets, we leveraged several high-quality existing datasets. The primary sources included The Pile[39], a large-scale, diverse dataset of text from various sources; Natural Questions, a dataset of real user queries and corresponding answers, which we augmented with longer contexts; BookCorpus, a collection of books that provided extended narratives; and scientific papers from arXiv, which were used to create examples with technical and specialized language.

4 Experiments

4.1 Testing Datasets

The testing dataset comprises 3,740 (context, prompt, response) samples derived from the Prompt-with-Context (PWC) dataset, introduced in the ICAE paper [14]. The original PWC dataset contains 240,000 samples for training and 18,000 samples for testing. We extracted 3,740 samples from the test set, selecting those with context lengths less than 512 words to align with the default maximum context length of the Squid model.

The questions in our dataset can be categorized into six types:

1. **Contextual QA**: Questions seeking specific facts without numeric values.
2. **Numeric QA**: Questions requesting numeric values and facts.
3. **Rephrasing**: Tasks asking to rewrite the given context.
4. **Summarization**: Tasks requiring summarization of the given context.
5. **Title / Keywords**: Tasks requesting a title or keywords for the given context.
6. **Continuation**: Tasks asking to write a continuation or follow-up paragraph to the given context.

Table 2 provides examples for each category of questions. Our testing experiments covers all six categories of questions in order to provide a comprehensive evaluation of the Squid model.

Table 2: Examples of question types in the testing dataset

Category	Count (Frequency)	Example Question
Contextual QA	2110 (56.36%)	"Explain the significance of Red Hat's acquisition of NooBaa."
Numeric QA	344 (9.19%)	"What is the overall length and diameter of the Stainless Phantom M2 .30 Cal. Sound Suppression System?"
Rephrasing	257 (6.86%)	"rephrase the above text"
Summarization	265 (7.08%)	"summarize the above text"
Title / Keywords	516 (13.78%)	"write a title for the above text" "extract a few keywords for the above text"
Continuation	252 (6.73%)	"write a paragraph (i.e., continuation) that follows the above text"

4.2 Restoration performance

We first evaluate the autoencoding performance of the Squid model during the initial training phase. This evaluation focuses on the model's ability to accurately restore text after compression, which is crucial for maintaining semantic integrity in downstream tasks. Table 3 presents a specific example of the ICAE performing text restoration. The example demonstrates the model's high restoration accuracy, with only a single word difference between the original and restored texts. Notably, the rare word "stuttered" is restored as "stalled," a semantically similar term that maintains the overall meaning of the passage.

This example illustrates the model's capability to accurately restore text while preserving semantic integrity, even when encountering less common vocabulary.

4.3 Compression Performance

In this section, we present a comprehensive evaluation of the Squid model's latency and compression quality compared to other candidate models. All experiments were conducted using a single NVIDIA A100 80GB GPU on the Microsoft Azure Cloud platform.

Table 3: Restoration example

Origin Context	Restoration
The old clock in the attic hadn't ticked in years, its hands frozen at 3:47. Dust settled on its ornate face, a silent testament to forgotten time. One stormy night, as lightning illuminated the cramped space, the clock suddenly sprang to life. Its gears groaned, protesting their long slumber. The hands began to spin wildly, whirling through days, months, years. Outside, the world blurred—seasons changed in seconds, buildings rose and fell, faces aged and renewed. When the hands finally stuttered to a stop, everything was still. The attic remained unchanged, but beyond its walls lay a world both familiar and strange, transformed by the clock's temporal dance.	The old clock in the attic hadn't ticked in years, its hands frozen at 3:47. Dust settled on its ornate face, a silent testament to forgotten time. One stormy night, as lightning illuminated the cramped space, the clock suddenly sprang to life. Its gears groaned, protesting their long slumber. The hands began to spin wildly, whirling through days, months, years. Outside, the world blurred—seasons changed in seconds, buildings rose and fell, faces aged and renewed. When the hands finally stalled to a stop, everything was still. The attic remained unchanged, but beyond its walls lay a world both familiar and strange, transformed by the clock's temporal dance.

To assess the latency of the Squid model, which incorporates two decoders—Qwen2-0.5B and Qwen2-7B, we employed the Qwen2-7B model as a baseline. The Squid model leverages Qwen2-0.5B to generate compression tokens, which are then passed along with the tokenized question prompt to Qwen2-7B. In contrast, the Qwen2-7B model processes the raw input directly without any form of compression.

Our experimental results reveal that the Squid model significantly outperforms the Qwen2-7B model in terms of latency. As shown in Table 5, the Squid model achieves an average latency of 4.32 seconds, which is approximately 4.79 times faster than the Qwen2-7B model's latency of 20.71 seconds. This substantial reduction in latency highlights the efficiency of our decoder-decoder architecture, particularly in handling long contexts with minimized computational overhead. The inclusion of a smaller 0.5B decoder for context compression is pivotal in reducing the input length for the primary 7B decoder, thereby optimizing the overall inference process.

Table 4: Latency benchmark

Metric	Value
Average inference time (s) by Squid	4.32s
Average inference time (s) by Qwen2-7B	20.71s
Improvement Factor	4.79×

Table 5: Compression quality benchmark

Category	Correctness (%)
Contextual QA	97.76%
Numeric QA	98.53%
Rephrasing	99.22%
Summarization	99.62%
Title / Keywords	100.00%
Continuation	100.00%
Weighted average	98.53%

For the compression quality benchmark, we utilized GPT-4[40] to evaluate the accuracy of the model responses, given the input prompt and question. The correctness scores across various question categories, as detailed in Table 4, further underscore the robustness of the Squid model in maintaining high accuracy while enhancing efficiency. Notably, the model achieves perfect accuracy rates of 100% in the 'Title / Keywords' and 'Continuation' categories, with high scores in other categories, such as 97.76% for 'Contextual QA' and 98.53% for 'Numeric QA'. It is particularly noteworthy that even for numeric questions, which have stringent accuracy requirements due to the necessity of correct values, our model achieves an accuracy rate exceeding 98%. These results demonstrate that

the Squid model not only excels in reducing latency but also preserves the semantic integrity and correctness of the generated outputs across a wide range of tasks.

We compare the Squid model with AutoCompressor [12], based on Llama-2-7b, and Qwen2-7B, the base model for Squid's decoder component (Table 6). Our evaluation suggests that AutoCompressor may overfit to its training datasets, while Squid shows consistent performance. Notably, despite using compressed tokens, Squid demonstrates comparable performance to Qwen2-7B, winning 23.6% of comparisons and tying 44.2%, for a combined win-tie rate of 67.8%. This parity is significant, as compression models typically show performance degradation. The results indicate that Squid's compression techniques effectively preserve, and potentially enhance, the capabilities of Qwen2-7B, while reducing computational requirements. This achievement underscores the efficacy of our approach in maintaining model performance despite information loss from token compression.

Table 6: Comparison with AutoCompressor

System 1	System 2	Win (%)	Lose (%)	Tie (%)	Win + Tie (%)
Squid	**AutoCompressor**	95.1	0.0	4.9	100.0
	Qwen2-7B	23.6	32.2	44.2	67.8

The Squid model's superior performance in both latency and accuracy underscores its potential for energy-efficient, on-device language modeling, especially in resource-constrained environments where balancing speed and accuracy is crucial. These findings suggest that our proposed architecture offers a promising solution for applications that demand rapid and accurate natural language processing capabilities.

5 Conclusion

In this paper, we introduced Squid, a novel decoder-decoder architecture designed for efficient processing of long contexts in on-device language models. By treating extended context as a distinct modality, Squid utilizes a compact 0.5B parameter decoder to distill contextual information into memory tokens, which are then processed by a larger 7B parameter decoder. Our experiments demonstrate that this approach achieves a 10-fold improvement in energy efficiency and a 5-fold reduction in latency compared to conventional methods, while maintaining high accuracy across various task categories.

Squid represents a significant advancement towards more sustainable and scalable language models for resource-constrained environments. Its multi-stage training process, comprising restoration training, continual training, and instruction fine-tuning, enables effective handling of diverse long-context tasks. Future work could explore further optimizations and adaptations of this architecture to other modalities or specialized domains.

References

[1] Georgi Gerganov. llama.cpp, 2023.

[2] MLC team. MLC-LLM, 2023.

[3] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, et al. Mediapipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*, 2019.

[4] PyTorch Team. executorch: A pytorch extension for dynamic task scheduling. `https://github.com/pytorch/executorch`, 2023. Accessed: 2024-08-24.

[5] BentoML. Openllm, 2024.

[6] Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. Powerinfer: Fast large language model serving with a consumer-grade gpu, 2023.

[7] Wei Chen and Zhiyuan Li. Octopus v2: On-device language model for super agent, 2024.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020.

[9] Wenhu Chen Ziyan Jiang, Xueguang Ma. Longrag: Enhancing retrieval-augmented generation with long-context llms. *arXiv preprint arXiv:2406.15319*, 2024.

[10] Wangsong Yin, Mengwei Xu, Yuanchun Li, and Xuanzhe Liu. Llm as a system service on mobile devices, 2024.

[11] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 34892–34916. Curran Associates, Inc., 2023.

[12] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3829–3846, Singapore, December 2023. Association for Computational Linguistics.

[13] Jesse Mu, Xiang Li, and Noah Goodman. Learning to compress prompts with gist tokens. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 19327–19352. Curran Associates, Inc., 2023.

[14] Tao Ge, Hu Jing, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *The Twelfth International Conference on Learning Representations*, 2024.

[15] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[16] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 26296–26306, June 2024.

[17] Ji Lin, Hongxu Yin, Wei Ping, Pavlo Molchanov, Mohammad Shoeybi, and Song Han. Vila: On pre-training for visual language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 26689–26699, June 2024.

[18] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond, 2023.

[19] Haoyu Lu, Wen Liu, Bo Zhang, Bingxuan Wang, Kai Dong, Bo Liu, Jingxiang Sun, Tongzheng Ren, Zhuoshu Li, Hao Yang, Yaofeng Sun, Chengqi Deng, Hanwei Xu, Zhenda Xie, and Chong Ruan. Deepseek-vl: Towards real-world vision-language understanding, 2024.

[20] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[21] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models, 2023.

[22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[23] Chengfei Lv, Chaoyue Niu, Renjie Gu, Xiaotang Jiang, Zhaode Wang, Bin Liu, Ziqi Wu, Qiulin Yao, Congyu Huang, Panos Huang, Tao Huang, Hui Shu, Jinde Song, Bin Zou, Peng Lan, Guohuan Xu, Fei Wu, Shaojie Tang, Fan Wu, and Guihai Chen. Walle: An End-to-End, General-Purpose, and Large-Scale production system for Device-Cloud collaborative machine learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 249–265, Carlsbad, CA, July 2022. USENIX Association.

[24] Le Xue, Ning Yu, Shu Zhang, Artemis Panagopoulou, Junnan Li, Roberto Martín-Martín, Jiajun Wu, Caiming Xiong, Ran Xu, Juan Carlos Niebles, et al. Ulip-2: Towards scalable multimodal pre-training for 3d understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 27091–27101, 2024.

[25] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. Hubert: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM transactions on audio, speech, and language processing*, 29:3451–3460, 2021.

[26] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, et al. Conformer: Convolution-augmented transformer for speech recognition. *arXiv preprint arXiv:2005.08100*, 2020.

[27] Sheng Shen, Liunian Harold Li, Hao Tan, Mohit Bansal, Anna Rohrbach, Kai-Wei Chang, Zhewei Yao, and Kurt Keutzer. How much can CLIP benefit vision-and-language tasks? In *International Conference on Learning Representations*, 2022.

[28] Pengchuan Zhang, Xiyang Dai, Jianwei Yang, Bin Xiao, Lu Yuan, Lei Zhang, and Jianfeng Gao. Multi-scale vision longformer: A new vision transformer for high-resolution image encoding. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 2998–3008, 2021.

[29] Ronak Pradeep, Sahel Sharifymoghaddam, and Jimmy Lin. Rankzephyr: Effective and robust zero-shot listwise reranking is a breeze! *arXiv preprint arXiv:2312.02724*, 2023.

[30] Ronak Pradeep, Sahel Sharifymoghaddam, and Jimmy Lin. Rankvicuna: Zero-shot listwise document reranking with open-source large language models. *arXiv preprint arXiv:2309.15088*, 2023.

[31] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with bert. *arXiv preprint arXiv:1901.04085*, 2019.

[32] Muhammad Asif Ali, Zhengping Li, Shu Yang, Keyuan Cheng, Yang Cao, Tianhao Huang, Lijie Hu, Lu Yu, and Di Wang. Prompt-saw: Leveraging relation-aware graphs for textual prompt compression. *arXiv preprint arXiv:2404.00489*, 2024.

[33] Fangyuan Xu, Weijia Shi, and Eunsol Choi. Recomp: Improving retrieval-augmented lms with compression and selective augmentation. *arXiv preprint arXiv:2310.04408*, 2023.

[34] Yucheng Li, Bo Dong, Chenghua Lin, and Frank Guerin. Compressing context to enhance inference efficiency of large language models. *arXiv preprint arXiv:2310.06201*, 2023.

[35] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. *arXiv preprint arXiv:2310.06839*, 2023.

[36] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023.

[37] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.

[38] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.

[39] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The Pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.

[40] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen

He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. Gpt-4 technical report, 2024.

[41] Howard Yen, Tianyu Gao, and Danqi Chen. Long-context language modeling with parallel context encoding. In *Association for Computational Linguistics (ACL)*, 2024.

[42] Sijun Tan, Xiuyu Li, Shishir Patil, Ziyang Wu, Tianjun Zhang, Kurt Keutzer, Joseph E. Gonzalez, and Raluca Ada Popa. Lloco: Learning long contexts offline. *arXiv preprint arXiv: 2404.07979*, 2024.

[43] Amanda Bertsch, Uri Alon, Graham Neubig, and Matthew R. Gormley. Unlimiformer: Long-range transformers with unlimited length input. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.

[44] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024.

[45] Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Richard James, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. REPLUG: Retrieval-augmented black-box language models. In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8371–8384, Mexico City, Mexico, June 2024. Association for Computational Linguistics.

[46] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

[47] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv:2004.05150*, 2020.

[48] Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507*, 2019.

[49] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. *arXiv preprint arXiv:2009.14794*, 2020.

[50] Lin Zheng, Chong Wang, and Lingpeng Kong. Linear complexity randomized self-attention mechanism. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 27011–27041. PMLR, 17–23 Jul 2022.

[51] Aydar Bulatov, Yuri Kuratov, and Mikhail Burtsev. Recurrent memory transformer. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022.

[52] Aydar Bulatov, Yuri Kuratov, Yermek Kapushev, and Mikhail S Burtsev. Scaling transformer to 1m tokens and beyond with rmt. *arXiv preprint arXiv:2304.11062*, 2023.